

Algorytmy Metaheurystyczne — Lista 2

Symulowane Wyżarzanie oraz Przeszukiwanie z Zakazami
dla Problemu Komiwożera

Maj 2026, Jędrzej Sajnog, indeks: 279701

1. Zadanie 1 — Symulowane Wyżarzanie (SA)

1.1. Opis problemu

Problem komiwożera (TSP — **Travelling Salesman Problem**) polega na znalezieniu najkrótszego cyklu Hamiltonowskiego w grafie pełnym: mając dane n miast o współrzędnych (x_i, y_i) , szukamy permutacji minimalizującej długość ścieżki.

W eksperymentach użyto instancji: wi29 (29 miast), dj38 (38), qa194 (194), uy734 (734), zi929 (929) do strojenia parametrów, a mu1979 (1979), ca4663 (4663), eg7146 (7146), ei8246 (8246) i tz6117 (6117) do testów.

1.2. Algorytm

Symulowane wyżarzanie: Na każdym kroku losowany jest ruch sąsiedzki (tu: **2-opt** — odwrócenie podsegmentu trasy). Ruch jest akceptowany zawsze, gdy poprawia rozwiązanie ($\Delta < 0$), a z prawdopodobieństwem $e^{\{-\frac{\Delta}{T}\}}$, gdy je pogarsza. Temperatura T jest stopniowo obniżana wg harmonogramu geometrycznego:

$$T_{\{k+1\}} = \alpha \cdot T_k, \quad \alpha \in (0, 1).$$

Schemat zatrzymania: SA działa od T_0 do $T \approx 1$, co daje $\lceil \log(1/T_0) / \log(\alpha) \rceil$ epok. W każdej epoce wykonywanych jest `iters_per_temp` prób ruchu.

Ruch 2-opt wybiera losowo dwa indeksy $i < j$ i odwraca segment trasy $[i + 1, j]$. Zmiana kosztu obliczana jest bez przeliczania całej trasy:

$$\Delta = d(c_{\{\pi(i)\}}, c_{\{\pi(j)\}}) + d(c_{\{\pi(i+1)\}}, c_{\{\pi(j+1)\}}) - d(c_{\{\pi(i)\}}, c_{\{\pi(i+1)\}}) - d(c_{\{\pi(j)\}}, c_{\{\pi(j+1)\}}).$$

1.3. Implementacja

Implementacja w C++ (`tsp_sa.h`, `main.cpp`, `main2.cpp`):

- `run_sa(cities, params, rng)` — pojedyncze uruchomienie SA; zwraca najlepszą trasę i koszt znaleziony w trakcie całego biegu.
- Parametry: `init_temp` T_0 , `cooling_rate` α , `iters_per_temp`.
- Inicjalizacja: losowa permutacja miast (shuffle z MT19937).
- Śledzenie najlepszego: zachowywana jest osobna kopia `best` aktualizowana przy każdym polepszeniu, niezależnie od bieżącej temperatury.

1.4. Dobór parametrów (grid search)

Grid search wykonano na instancjach wi29–zi929 (jeden przebieg SA na kombinację). Przeszukiwana siatka:

Parametr	Wartości
T_0	1000, 5000, 6000, 10 000
α	0.85, 0.9, 0.985, 0.990
iters_per_temp	1000, 2000, 5000, 7000, 10 000

Chociaż nie naraz (za dużo możliwych kombinacji). Zamiast tego testowano mniejszą liczbę kombinacji i na podstawie najlepszej decydowano się na zmianę przeszukiwanych wartości.

Kryterium oceny — suma względnych kosztów:

$$\text{score} = \sum_{i=1}^{\{N_{\text{inst}}\}} \frac{c_i}{c_i^*},$$

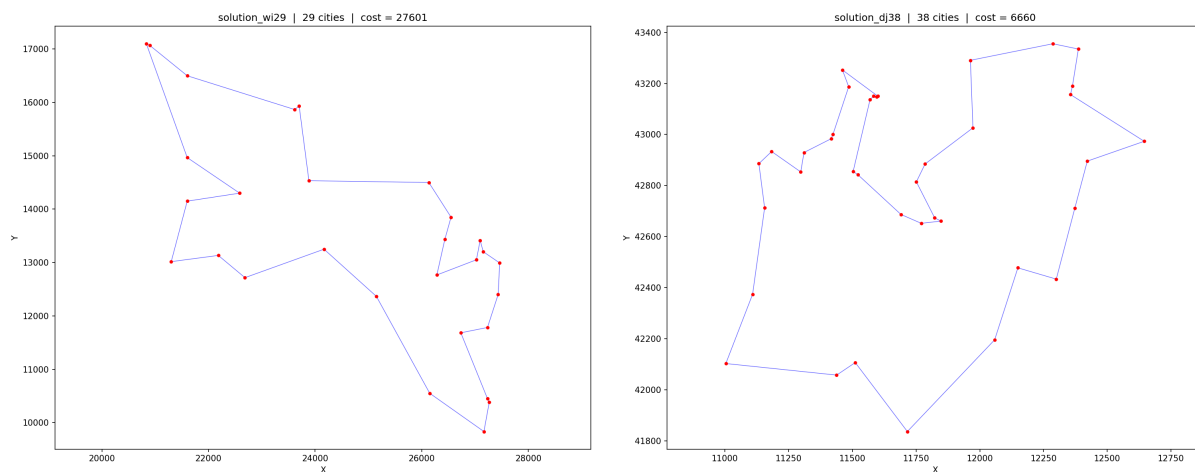
gdzie c_i^* to najlepszy koszt uzyskany przez **jakąkolwiek** kombinację na instancji i . Niższa wartość = lepiej.

Najlepsze parametry: $T_0 = 10000$, $\alpha = 0.99$, iters_per_temp = 10000.

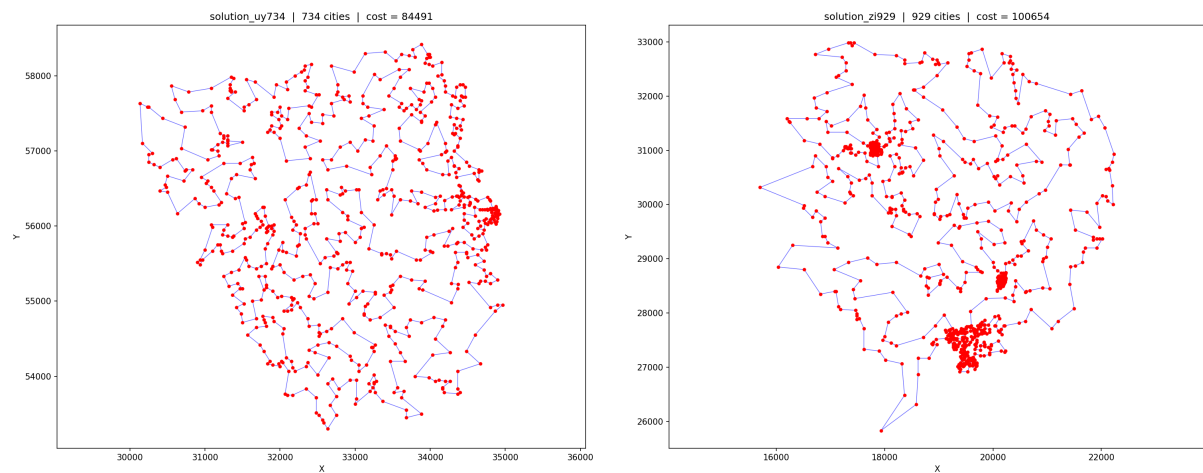
Wyniki dla poszczególnych instancji z najlepszymi parametrami:

Instancja	n	Koszt SA
wi29	29	27 601
dj38	38	6 659
qa194	194	9 540
uy734	734	85 297
zi929	929	104 339

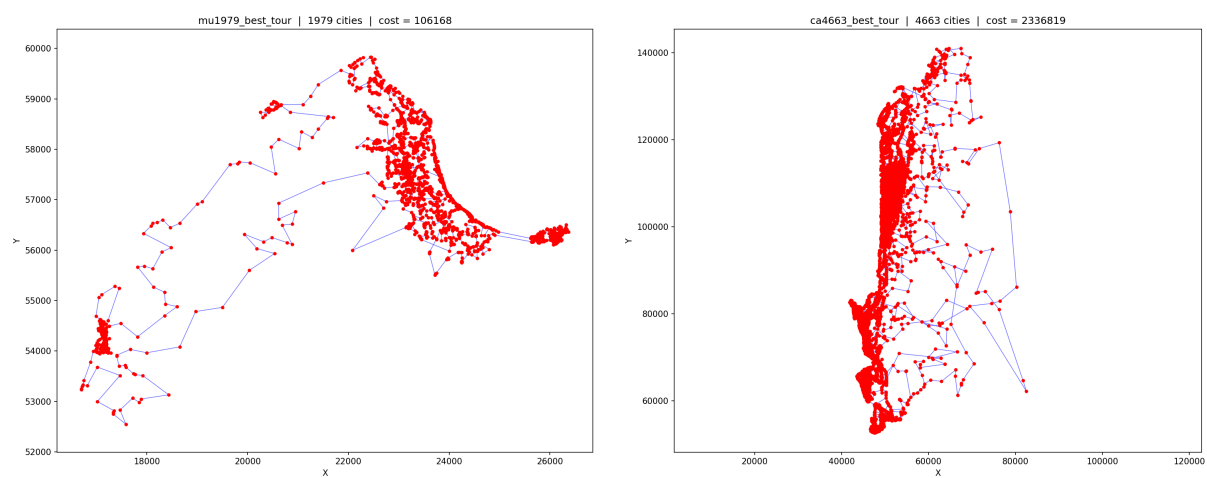
1.5. Wykresy



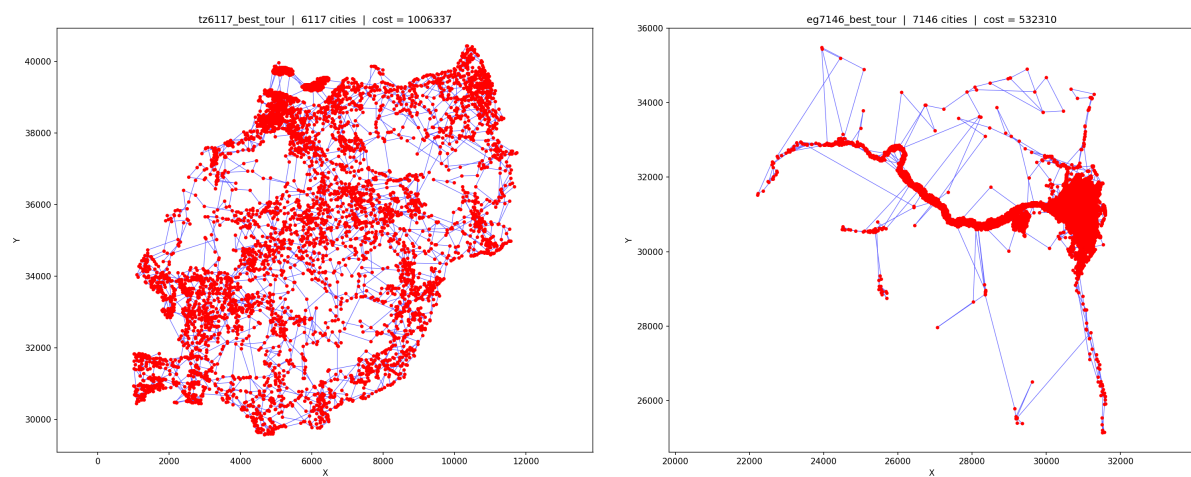
Rysunek 1: Trasy SA dla wi29 i dj38.



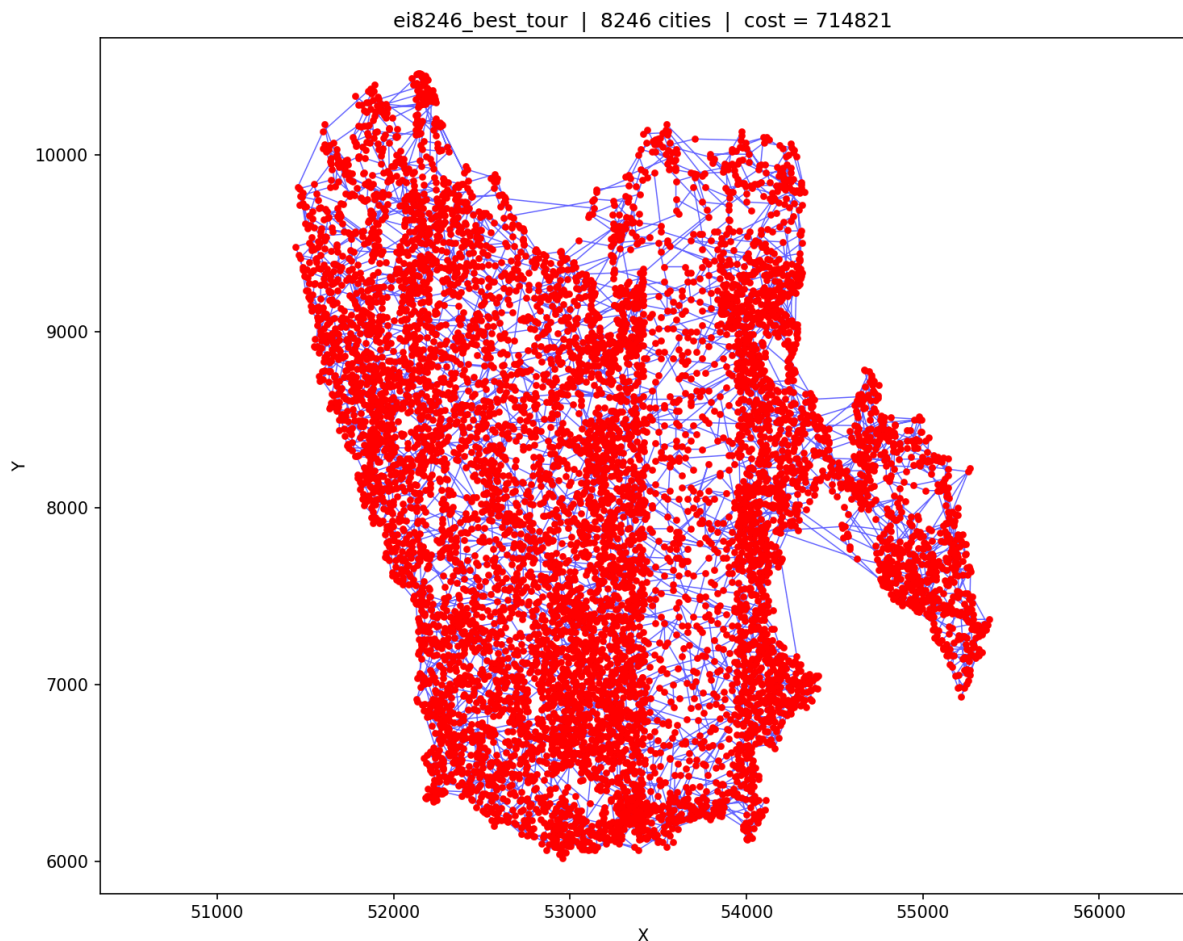
Rysunek 2: Trasy SA dla uy734 i zi929.



Rysunek 3: Trasy SA dla mu1979 i ca4663.



Rysunek 4: Trasy SA dla tz6117 i eg7146.



Rysunek 5: Trasa SA dla ei8246.

1.6. Obserwacje i wnioski

- **Rola α i `iters_per_temp`:** Wolniejsze chłodzenie ($\alpha = 0.990$) oraz więcej iteracji na temperaturę konsekwentnie daje lepsze wyniki. Wyższe T_0 lekko przeważa nad niższym przy tych samych pozostałych parametrach, możliwie ze względu na większą liczbę iteracji.
- **Porównanie z poprzednimi listami:** Wyniki są około 50% gorsze w porównaniu z poprzednimi metodami, szczególnie dla dużej liczby miast.
- **Rozwiązania:** SA dobrze znajduje „główny wygląd” ścieżki, ale ma problemy ze znalezieniem dokładniejszego rozwiązania.

2. Zadanie 2 — Przeszukiwanie z Zakazami (TS)

2.1. Algorytm

Przeszukiwanie z zakazami (**Tabu Search**) to algorytm lokalnego przeszukiwania z pamięcią krótkoterminową. W każdej iteracji wybierany jest **najlepszy** dopuszczalny ruch spośród próbki sąsiedztwa — nawet jeśli pogarsza on bieżące rozwiązanie. Zapobiega to zapętleniom dzięki **liście tabu**: ostatnio wykonane ruchy są tymczasowo zakazane.

Kryterium aspiracji: ruch tabu jest dopuszczalny, jeśli prowadzi do nowego globalnego minimum (przebicie bieżącego tabu).

Warunek stopu: brak poprawy globalnego minimum przez `max_no_improve` kolejnych iteracji.

2.2. Implementacja

Implementacja w (tsp_ts.h, main.cpp, main2.cpp):

- Lista tabu przechowuje pary (a, b) z $a < b$ — wewnętrzne końcowe indeksy odwróconego segmentu.
- W każdej iteracji losowane jest `neighbor_sample` kandydatów 2-opt; wybierany jest ten o najniższym Δ wśród dozwolonych lub spełniających kryterium aspiracji.
- Parametry: `tabu_size`, `neighbor_sample`, `max_no_improve`.
- Inicjalizacja: ta sama co w SA — losowa permutacja.

2.3. Dobór parametrów (grid search)

Parametr	Wartości
<code>tabu_size</code>	10, 50, 100
<code>neighbor_sample</code>	50, 200, 500
<code>max_no_improve</code>	300, 1000, 3000

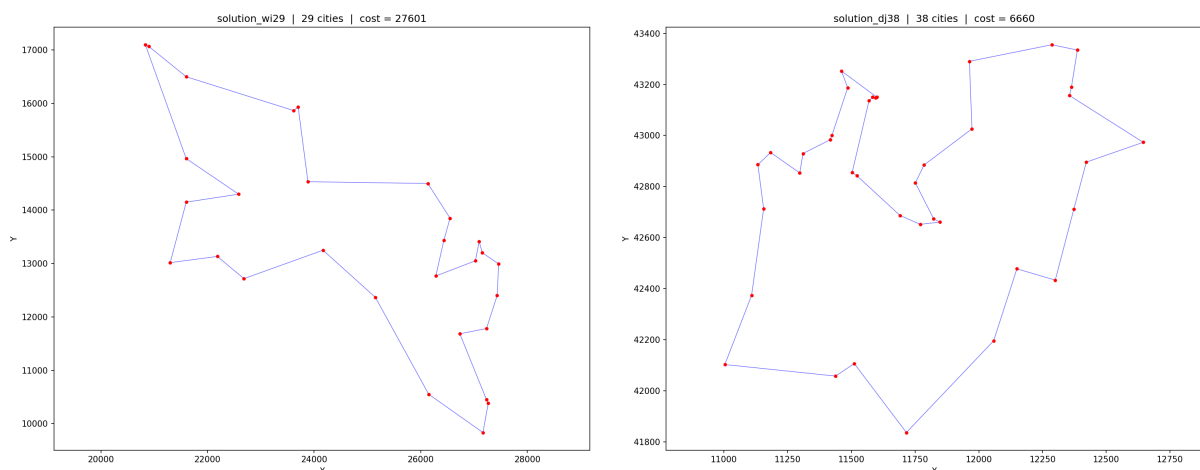
Ta sama metryka score co w SA.

Najlepsze parametry: `tabu_size= 10`, `neighbor_sample= 500`, `max_no_improve= 3000`.

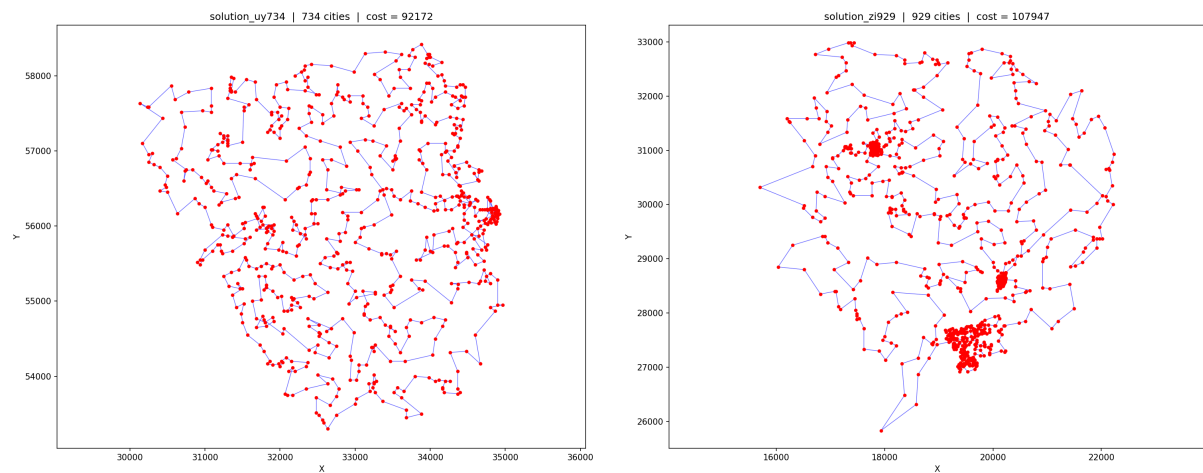
Wyniki dla poszczególnych instancji:

Instancja	n	Koszt TS
wi29	29	27 601
dj38	38	6 659
qa194	194	9 875
uy734	734	93 513
zi929	929	104 514

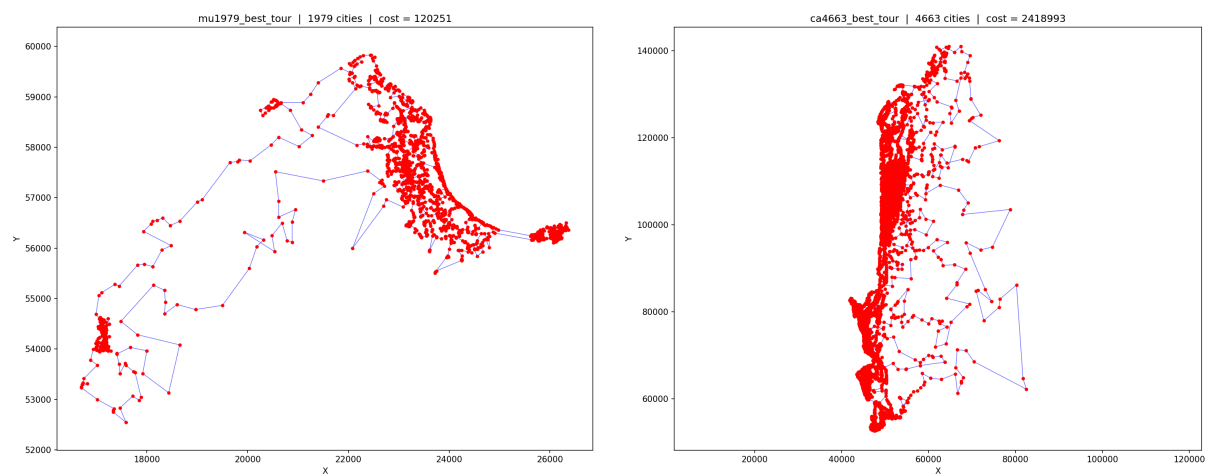
2.4. Wykresy



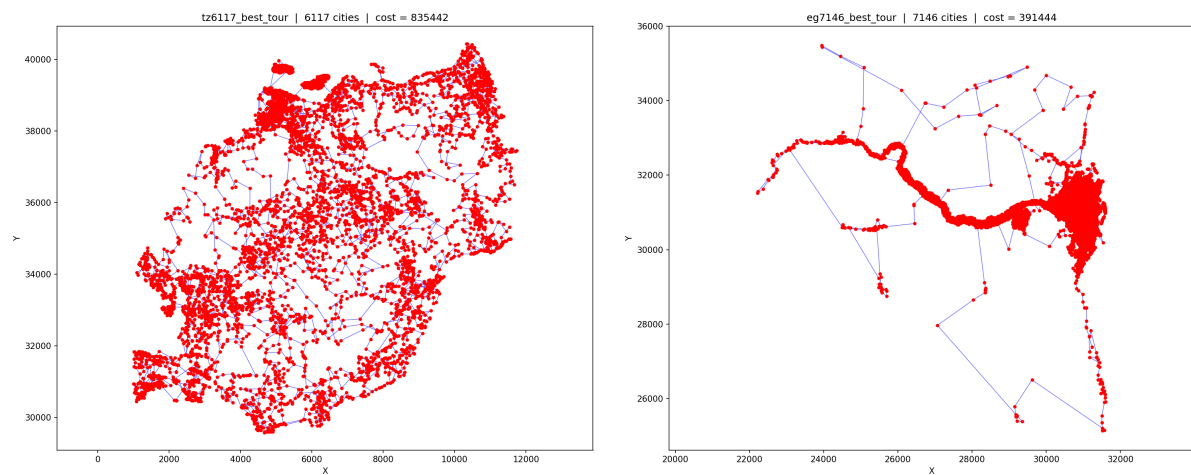
Rysunek 6: Trasy TS dla wi29 i dj38 — najlepsze parametry.



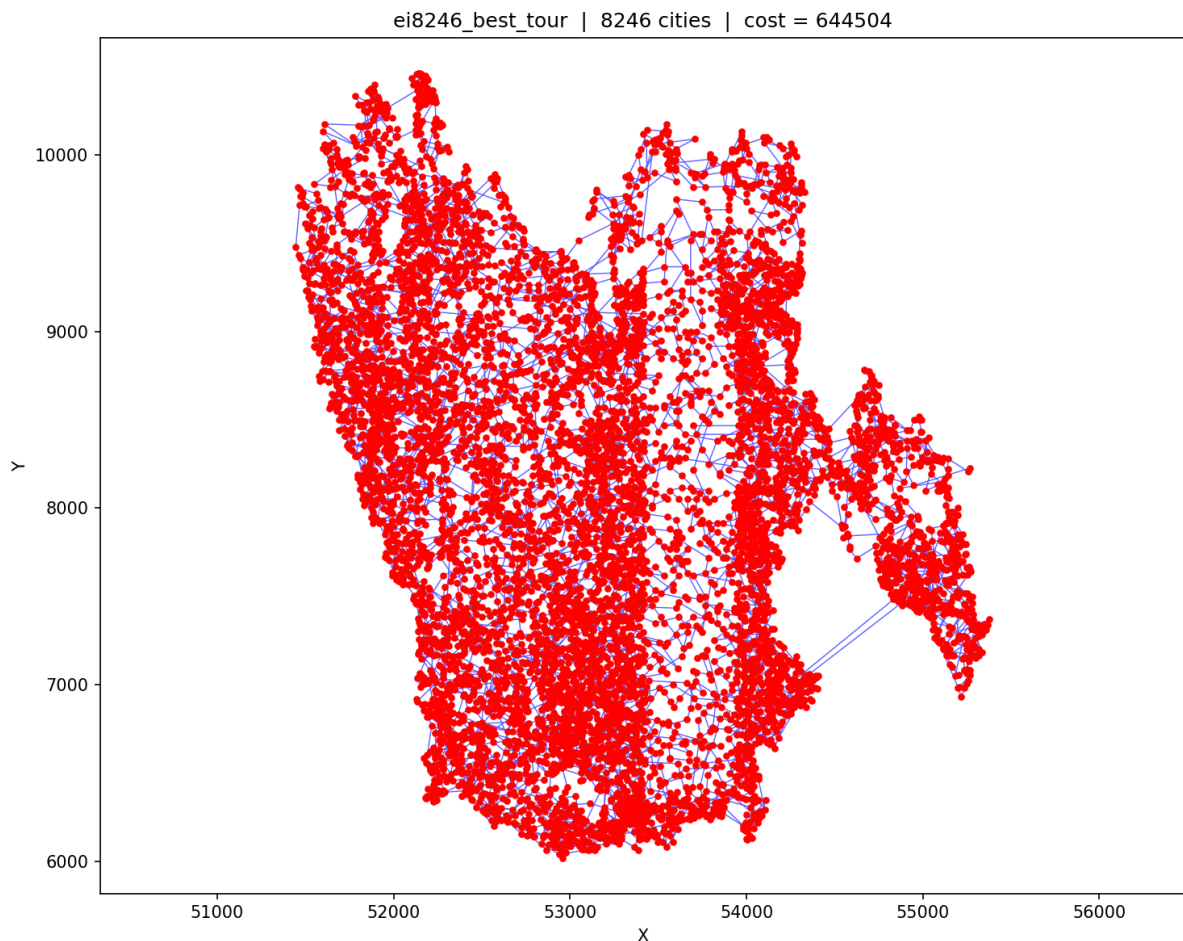
Rysunek 7: Trasy TS dla uy734 (734 miast) i zi929 (929 miast).



Rysunek 8: Trasy SA dla mu1979 i ca4663.



Rysunek 9: Trasy SA dla tz6117 i eg7146.



Rysunek 10: Trasa SA dla ei8246.

2.5. Obserwacje i wnioski

- **Kluczowy wpływ `neighbor_sample`:** Wyniki z `sample=50` są dramatycznie gorsze niż z `sample=500`. Ocena zbyt małej próbki sąsiedztwa prowadzi do wyborów praktycznie losowych.
- **Małe `tabu_size` najlepsze:** `tabu_size=10` wygrywa we wszystkich konfiguracjach z `sample=500`. Zbyt duża lista tabu nadmiernie ogranicza przestrzeń dopuszczalnych ruchów.
- **Zmienność wyników:** Wyniki mają dosyć wysoką rozpiętość (zdjęcia osobno).

3. Porównanie algorytmów

3.1. Wnioski końcowe

SA osiąga lepsze wyniki jakościowe TS daje lepsze wyniki od SA szczególnie na dużych instancjach problemu. Na mniejszych instancjach osiągają porównywalne wyniki.

Porównanie z poprzednimi listami TS daje wyniki porównywalne z poprzednimi podejściami np. z MST. Chociaż zwykle daje wyniki lekko gorsze. SA daje wyniki znacząco gorsze.

Wrażliwość na parametry: SA jest stosunkowo mało wrażliwy — różnice między najlepszą a najgorszą kombinacją wynoszą na małych instancjach. TS jest bardzo wrażliwy na `neighbor_sample` — zła wartość (50) jest gorsza o >40% od optymalnej (500).

Ograniczenia SA: przy dużych instancjach ($n > 4000$) zadana liczba iteracji nie wystarcza, wyniki dla ca4663 i większych.